EcoCycle Base para elaboração do relatório técnico

Documento de contexto para redação assistida por inteligência artificial

Data de referência: 29 de outubro de 2026. Base: código e documentação presentes na pasta do projeto nesta sessão. Este documento descreve o estado observado; não substitui um relatório de ensaio físico nem uma especificação do backend.

1 Finalidade e escopo

Use este material como esqueleto factual para escrever o relatório técnico do EcoCycle, um protótipo de esteira inteligente que identifica peças por cor e solicita o posicionamento de recipientes. O computador processa imagens, gerencia lotes e comunica-se com uma ESP32. A placa controla o motor da esteira e um servo posicional SG90 de 180 graus.

O objetivo funcional é executar o ciclo identificar uma peça, parar a esteira, posicionar o recipiente e retomar o transporte. A operação depende da seleção de um lote e do clique explícito em Iniciar triagem. A visão usa cores como representação de materiais; não verifica a composição física dos objetos.

Situação das funcionalidades

Recurso	Situação no projeto
Captura de câmera IP e detecção por cor	Implementado com OpenCV, ROI ajustável, filtro de fundo e rastreamento por centroide.
Seleção e operação de lote	Implementado com catálogo local, início explícito, pausa e conclusão.
Comunicação com a ESP32	Implementada por WebSocket, com JSON, sessão, comandos, ACKs e telemetria.
Acionamento dos motores	Previsto no firmware atual para motor TT por L9110S e servo SG90 posicional. O código não comprova o movimento físico.
Peso, temperatura, força e velocidade	Simulados. Não são medições de sensores.
Backend externo e persistência	Integração futura. Não há contrato externo definido nem banco de dados implementado nesta versão.

Regras para a redação

Descrever funcionalidades implementadas como comportamento do software. Apresentar integração com backend e sensores reais como evolução prevista. Não afirmar precisão, produtividade, economia de energia, taxa de acerto ou capacidade de carga sem ensaio documentado. Campos acadêmicos e resultados ausentes devem permanecer como [PREENCHER PELA EQUIPE].

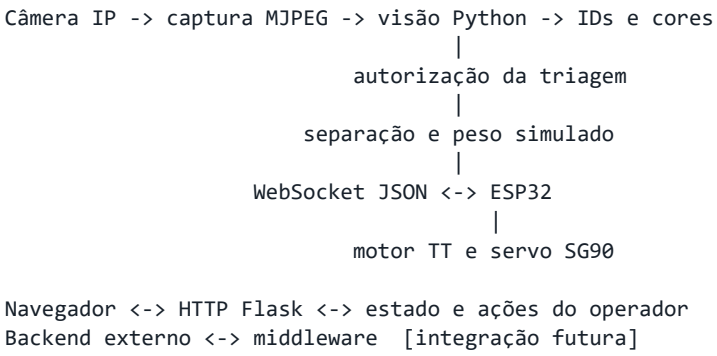
2 Arquitetura e componentes

Distribuição das responsabilidades

O navegador consulta a aplicação Flask para apresentar os lotes e o dashboard. Os botões enviam ações HTTP ao middleware Python. O navegador não controla GPIOs nem inicia a triagem automaticamente ao abrir a página.

No computador, a captura da câmera alimenta a visão. A visão detecta peças e envia observações aos controladores de separação e peso. O controlador de triagem autoriza ou bloqueia a operação. O WebSocket transmite comandos à ESP32 e recebe confirmações e estado do acionamento.

Na ESP32, o firmware mantém uma máquina de estados para executar a sequência de parada, posicionamento e retomada. O transporte depende de conexão, hardware habilitado, autorização do operador e ausência de bloqueio térmico.



Componentes da montagem informada

Componente	Papel no protótipo
Celular com IP Webcam ou câmera IP compatível	Fornecer vídeo ao computador pela rede.
Computador com Python	Interpretar imagens, hospedar a página e coordenar a operação.
ESP32	Receber comandos e aplicar sinais nos atuadores. O modelo exato da placa deve ser informado pela equipe.
Motor TT e módulo L9110S	Movimentar a esteira e receber comandos de ligar/parar.
Servo SG90 posicional de 180 graus	Solicitar as três posições dos recipientes.
Fontes e estrutura mecânica	Alimentação, correia, roletes e suporte dos recipientes precisam ser descritos conforme a montagem real.

O usuário informou alimentação externa de aproximadamente 5 V para o servo. O relatório deve confirmar a montagem efetivamente apresentada, o GND comum e as fontes utilizadas, sem inventar corrente, dimensões ou capacidade de carga.

3 Estrutura do software

Módulos principais

Arquivo	Responsabilidade
config.py	Centraliza rede, GPIOs, curso do servo, tempos, lotes e parâmetros de visão e simulação.
app.py	Ponto de entrada Flask, páginas, API HTTP, vídeo e inicialização da captura.
triage.py	Seleção, início, pausa, retomada e conclusão da operação por lote.
vision.py	Captura, decodificação, ROI, detecção, rastreamento e publicação do vídeo anotado.
mjpeg.py	Recompõe JPEGs de pacotes fragmentados e recupera o fluxo após imagens incompletas.
camera_buffer.py	Compartilha somente o último quadro pendente entre captura e processamento.
background.py	Aprende o fundo vazio e ignora regiões inicialmente coloridas.
sorting.py	Confirma IDs e cria um único comando ativo de separação, sem fila de materiais.
websocket_server.py	Expõe monitoramento e o canal exclusivo de comandos para a ESP32.
batch_weights.py	Simula pesos por peça, subtotais por material, saldo e fechamento do lote.
temperature.py	Simula aquecimento e resfriamento e mantém a trava térmica.
motion.py	Calcula força fictícia e velocidade visual a partir do peso processado.
generate_esp32_config.py	Exporta configurações Python para os headers C++ dos sketches.

Interface e firmware

```
static/{lotes.html,index.html}    seleção e dashboard
esp32/EcoCycleClient/
  EcoCycleClient.ino             firmware principal
  EcoCycleConfig.h               configuração gerada
esp32/TesteServo/
  TesteServo.ino                  diagnóstico isolado
  EcoCycleConfig.h               sem credenciais
tests/
  test_*.py                     testes Python
  firmware/test_cycle.cpp       cenários do sketch real
  firmware/fakes/*.h            periféricos fictícios para teste
```

O CSS e o JavaScript ficam dentro dos próprios arquivos HTML. requirements.txt declara Flask, flask-sock, OpenCV, NumPy e requests. GUIA_CODIGO.md orienta a leitura dos fontes; SIMULACAO_PESOS.md detalha as regras de peso. Os arquivos .h são gerados a partir de config.py; alterações destinadas à placa exigem gerar, compilar e regravar o firmware.

4 Fluxo operacional e visão computacional

Sequência da operação

1. Executar app.py, iniciar a captura e registrar o fundo com câmera fixa e esteira vazia.
2. Abrir a tela inicial e selecionar um lote disponível. A seleção não liga o motor.
3. No dashboard, clicar em Iniciar triagem. A API verifica câmera, fundo e condições da operação.
4. Autorizar a ESP32 pelo WebSocket. A placa só transporta se seus bloqueios permitirem.
5. Detectar uma nova peça dentro da ROI e confirmar a cor em quadros consecutivos.
6. Criar o comando para uma cor com destino e contabilizar separadamente o peso simulado.
7. Parar a esteira, aguardar acomodação, aplicar o pulso do servo e esperar o posicionamento.
8. Tentar retomar a esteira e enviar ACK. Pausa, calor ou desconexão podem impedir a retomada.
9. Ao fechar o peso do lote, aguardar o ciclo ativo e o transporte final antes de pausar a operação.

Etapas da interpretação da imagem

A captura recebe bytes MJPEG pela rede. mjpeg.py separa JPEGs completos, respeitando marcadores e comprimentos de segmentos. A leitura e a análise ocorrem em rotinas separadas. O buffer mantém apenas o último quadro pendente para evitar acúmulo de imagens antigas; isso não mede nem elimina atrasos internos do celular e da rede.

O quadro é redimensionado para 640 por 480 pixels. A ROI é definida em proporções e pode ser movida ou redimensionada pelo navegador. Na região selecionada, a visão converte BGR para HSV, aplica intervalos de cor e operações morfológicas, encontra contornos e descarta candidatos pequenos ou muito finos.

O filtro de fundo aprende a cena vazia e reserva as regiões inicialmente coloridas. Depois compara os quadros com a referência congelada. Assim, uma peça nova parada durante o movimento do servo não é absorvida automaticamente pelo fundo. Mover a câmera ou mudar o cenário pode exigir recaptura.

O rastreador associa centros próximos da mesma cor e reaproveita IDs recentes. IDs são temporários: oclusão, contato entre peças ou saída e retorno à ROI podem produzir associação incorreta ou um novo ID. Não há rede neural, treinamento de modelo ou reconhecimento físico de material implementado.

Associação atual entre cores e destinos

Cor	Material representado	Destino do servo
Laranja	Plástico	Recipiente A em 0 graus.
Verde claro	Alumínio	Recipiente B em 90 graus.
Azul claro	Metal	Recipiente C em 180 graus.
Roxo	Rejeito	Continua identificado e contabilizado, sem comando de posicionamento.

5 Simulações e gerenciamento do lote

Peso por material

Cada peça confirmada recebe um peso fictício em gramas inteiras. O controle usa eventoId para impedir duplicação de um evento e IDs da câmera para não pesar novamente a mesma peça rastreada. Os pesos são atribuídos na identificação; não comprovam a queda no recipiente ou a execução física do comando.

Categoria	Material exibido	Faixa normal por peça
PLASTICO	Plástico	100 a 300 g
OUTRO_METAL	Alumínio	200 a 600 g
METAL	Metal	300 a 700 g
REJEITO	Rejeito	50 a 250 g

```
saldo = total_do_lote - peso_processado
se saldo < minimo_do_material:
    peso_da_peca = saldo
    tipoCalculo = AJUSTE_SALDO
senao:
    peso_da_peca = sorteio_inteiro(minimo, min(maximo, saldo))
    tipoCalculo = ALEATORIO
```

Depois de aceitar a peça, o sistema atualiza peso processado, subtotal e quantidade da categoria. O lote recebe CONCLUIDO ao atingir exatamente o total. No exemplo de um lote de 1.000 g com 920 g já processados, uma peça de plástico recebe os 80 g restantes por ajuste de saldo.

O catálogo atual está em AVAILABLE_BATCHES. Na leitura desta versão, os totais cadastrados são 50.000.000 g, 30.000.000 g e 100.000.000 g. São valores de configuração da simulação e não capacidade física comprovada. Para um exemplo didático no relatório, pode-se usar 1.000 g, identificando-o como exemplo.

Temperatura e velocidade

A temperatura parte de 25 graus Celsius, cresce 0,15 grau por segundo de funcionamento informado e esfria 0,10 grau por segundo parada, limitada ao intervalo de 25 a 70 graus. Telemetria vencida passa a simulação para resfriamento. Ao alcançar 60 graus, arma a trava; a liberação manual exige 50 graus ou menos e confirmação recente de parada térmica da ESP32.

A força fictícia usa o peso processado em quilogramas multiplicado por 2 N por kg. A velocidade visual usa a referência de 0,10 m/s dividida por um mais a razão entre a força calculada e a referência de 2 N. A 1.000 g processados, o modelo resulta em 2 N e 0,05 m/s quando o motor está informado como ligado. Parado confirmado mostra zero; sem estado recente, mostra valor desconhecido. Essa fórmula não controla a velocidade do motor.

6 Interface HTTP e comunicação com o backend

Rotas implementadas no middleware

Método e rota	Função
GET /	Exibir a seleção de lotes.
GET /dashboard	Exibir o painel somente para o lote selecionado correspondente.
GET /api/lotes	Retornar lotes locais e permissões de seleção.
POST /api/lotes/selecionar	Receber loteId e preparar o lote sem iniciar o motor.
POST /api/triagem/iniciar	Receber loteId e solicitar início ou retomada explícita.
POST /api/triagem/pausar	Retirar a autorização sem apagar o progresso.
GET /api/detections	Reunir visão, peso, temperatura, velocidade e triagem.
POST /api/roi	Atualizar x, y, w e h normalizados da região analisada.
POST /api/camera/background	Agendar recaptura do fundo.
POST /api/conveyor/thermal/reset	Solicitar liberação térmica manual.
GET ou POST /api/lote	Consultar ou cadastrar um lote para simulação local.
POST /api/lote/pecas	Simular evento de peça manualmente, sem acionar motores.
GET /video	Transmitir o vídeo anotado em HTTP multipart JPEG.

O dashboard consulta o estado periodicamente, em intervalo nominal de 500 ms após concluir a consulta anterior. O vídeo usa uma requisição separada. A existência da rota WebSocket de monitoramento não significa que a página a utilize para renderizar seus indicadores.

Contratos locais mínimos

```
{"loteId":"EXEMPLO-001","pesoTotalLoteGramas":1000}
```

O objeto acima é aceito por POST /api/lote. Para eventos manuais de simulação, POST /api/lote/pecas recebe eventoId e material, por exemplo PLASTICO. Esse cadastro não autoriza a esteira; a operação exige seleção e Iniciar triagem. Dados inválidos e conflitos são recusados, normalmente com HTTP 400 ou 409 conforme a rota.

Integração externa prevista

O backend deverá fornecer identificador e peso total dos lotes. O middleware poderá devolver progresso, pesos por material e status. URL, tecnologia, autenticação, periodicidade, persistência e contrato externo ainda precisam ser definidos. Não apresentar as rotas locais como endpoints já existentes nesse backend. Tratar o fluxo backend para middleware e retorno dos resultados como proposta de integração.

7 Protocolo WebSocket com a ESP32

Há duas rotas na mesma aplicação. /ws/detections publica retratos do estado para observadores. /ws/sorter aceita uma única conexão de acionamento e troca permissões, ciclos, ACKs e telemetria. O canal não mantém fila de objetos; existe somente o ciclo ativo.

Identificação e autorização

A placa anuncia suporte ao protocolo 2 antes de receber a sessão do servidor. O servidor usa esse handshake para recusar firmware que poderia iniciar sem autorização explícita. Exemplo ilustrativo de troca inicial:

```
{"type":"client_hello","protocol":2,"operator_start":true}
{"type":"hello","session_id":"sessao-exemplo",
 "thermal_blocked":false,"triage_enabled":false}
{"type":"triage_control","session_id":"sessao-exemplo",
 "enabled":true}
```

Comando de separação

O exemplo abaixo representa uma peça verde confirmada. Os IDs são ilustrativos. stop_ms e settle_ms são durações em milissegundos; angle_deg é o destino absoluto do servo posicional.

```
{
  "type": "sort_cycle",
  "session_id": "sessao-exemplo",
  "command_id": "comando-exemplo",
  "track_id": 7,
  "color": "Verde-claro",
  "container": "B",
  "material": "Alumínio",
  "angle_deg": 90,
  "stop_ms": 200,
  "settle_ms": 1500
}
```

Retorno e bloqueio

```
{"type":"ack","session_id":"sessao-exemplo",
 "command_id":"comando-exemplo","status":"done",
 "conveyor_running":true,"thermal_blocked":false,
 "triage_enabled":true}
{"type":"conveyor_state","session_id":"sessao-exemplo",
 "running":true,"thermal_blocked":false,"triage_enabled":true}
{"type":"thermal_control","session_id":"sessao-exemplo",
 "blocked":true}
```

ACK done confirma a conclusão da sequência de software. A telemetria informa o estado declarado do acionamento, sem sensor físico de giro. Sessão e command_id evitam aceitar respostas de outro ciclo. Na reconexão, o último comando concluído ou o ativo pode ser reconhecido sem repetir movimento; essa memória não sobrevive ao reinício da placa. A ausência de ACK até o prazo cancela o ciclo no servidor e encerra a conexão.

8 Firmware e parâmetros de operação

Máquina de estados

IDLE é o estado livre. STOPPING_CONVEYOR aguarda a acomodação após mandar parar. POSITIONING_CONTAINER aguarda depois de aplicar o pulso do servo. O loop usa diferenças de millis para avançar sem bloquear o atendimento do WebSocket durante o ciclo.

startConveyor só liga quando há hardware habilitado, PWM inicializado, sessão válida, autorização de triagem, ausência de trava térmica e estado IDLE. A desconexão para o transporte. Um posicionamento já em andamento pode terminar, mas isso não autoriza uma retomada bloqueada.

O firmware valida ângulo e tempos, trata duplicatas e rejeita instruções inválidas. Os motivos aparecem no Monitor Serial. Uma rejeição não deve interromper um ciclo válido nem deixar o transporte irregular por si só.

Valores observados na configuração

Parâmetro	Valor atual	Significado
SERVO_PIN	GPIO 18	Sinal do SG90.
CONVEYOR_PIN_IA e IB	GPIO 26 e GPIO 25	Entradas do L9110S, nesta ordem.
SERVO_TRAVEL_DEG	180 graus	Curso configurado para o servo posicional.
SERVO_MIN_US e MAX_US	500 e 2.500 microssegundos	Extremos de pulso configurados, sujeitos à calibração física.
SORT_CONFIRM_FRAMES	3 quadros	Confirmação da cor por ID.
SORT_STOP_MS	200 ms	Espera após mandar parar e antes de mover o servo.
SORT_SETTLE_MS	1.500 ms	Espera após aplicar o pulso de posicionamento.
SORT_ACK_TIMEOUT_SECONDS	15 s	Prazo de confirmação do ciclo.
BATCH_FINISH_RUN_SECONDS	3 s	Transporte final antes de pausar o lote.
ESP32_HANDSHAKE_TIMEOUT_SECONDS	3 s	Prazo para identificação inicial da placa.

Os valores são parâmetros do código, não medições de tempo mecânico, torque ou alinhamento. Calibrar conforme a montagem. O acionamento atual da esteira é liga/desliga, sem regulação de velocidade por PWM. O SG90 permanece responsável pelos recipientes; o MG995 de rotação contínua foi discutido como alternativa, mas não corresponde ao acionamento da esteira implementado nesta versão.

TesteServo.ino é um firmware temporário para diagnóstico sem Wi-Fi ou câmera. Com curso de 180 graus, solicita 90, 60 e 120 graus em sequência, mantendo as entradas da esteira baixas. Regravar o cliente principal para voltar à operação completa.

9 Evidências e limitações

Verificações disponíveis

A última execução de validação registrada nesta sessão, em 29 de outubro de 2026, terminou com 108 testes Python aprovados. Também foi verificada a sintaxe do JavaScript das duas páginas. A comparação estrutural da alteração de comentários preservou a lógica executável e os valores de configuração. Isso é evidência de software, não de desempenho físico.

Grupo de arquivos	Cobertura principal
test_background.py e test_mjpeg.py	Fundo, cores novas, ruído, ROI, fragmentação e recuperação de JPEG.
test_camera_buffer.py e test_sorting.py	Último quadro, atraso, criação de comando, confirmação, duplicata e ACK.
test_batch_weights.py	Faixas, saldo final, fechamento, duplicatas, concorrência e API local.
test_temperature.py e test_thermal_motion.py	Tempo térmico, expiração de estado, liberação manual e velocidade fictícia.
test_triage.py e test_websocket.py	Autorização explícita, seleção, pausa, conclusão e comunicação local.
test_config_export.py	Validação e geração das constantes da ESP32.
tests/firmware/test_cycle.cpp e fakes	Cenários do sketch real com periféricos substituídos. A existência desses testes não é prova de execução bem-sucedida em hardware.

Limitações que devem constar no relatório

- As cores representam categorias. Objetos novos com cores semelhantes podem ser aceitos se passarem pelos filtros; não há classificação da composição nem do formato específico de carimbinhos.
- IDs podem mudar após perda de rastreamento. Peças devem estar espaçadas; uma peça que passa durante indisponibilidade pode não ser recuperada, pois não existe fila.
- ROI, iluminação e referência de fundo influenciam o resultado. Regiões inicialmente ignoradas também podem ocultar uma peça colocada sobre elas.
- Peso simulado é contabilizado na detecção, e não após comprovação de entrega. Rejeito é contado, mas não tem posicionamento próprio no SG90 atual.
- Telemetria, PWM ativo e ACK não comprovam giro, frenagem ou alinhamento físicos. A trava térmica usa temperatura fictícia e não protege por medição real de aquecimento.
- Lotes, totais, sessões e IDs ficam em memória. Não há histórico persistido. Reiniciar o Python reinicia os dados e exige nova seleção e início.
- O backend externo não está integrado. Não há resultado medido de latência, acurácia, produtividade ou capacidade de carga disponível para fundamentar números no relatório.

As redes e credenciais reais não devem ser transcritas no relatório. Usar nomes genéricos, endereços ilustrativos e um diagrama lógico da comunicação.

10 Estrutura sugerida do relatório técnico

1. Identificação do trabalho. Nome do projeto, integrantes, curso, instituição, disciplina, orientador, local e data. Preencher com os dados reais da equipe.
2. Resumo e palavras chave. Sintetizar problema, objetivo, arquitetura do protótipo, método de identificação e limites dos resultados disponíveis.
3. Introdução. Contextualizar a separação automatizada e delimitar o uso de cores como representação didática dos materiais.
4. Objetivos. Geral: implementar o fluxo de triagem por cor. Específicos: capturar imagens, ajustar ROI, rastrear peças, operar lotes, comunicar com ESP32 e apresentar indicadores simulados.
5. Fundamentação técnica. Explicar visão por HSV, contornos, rastreamento, HTTP, WebSocket, JSON, ESP32 e controle posicional. Usar referências primárias verificadas; não inventar bibliografia.
6. Materiais e métodos. Descrever montagem real, fontes, conexões, rede, bibliotecas, organização do software e procedimento de calibração.
7. Desenvolvimento. Apresentar captura, fundo, detecção, autorização por lote, ciclo do firmware, protocolos e modelos de simulação.
8. Testes e resultados. Separar testes automatizados, demonstrações observadas e ensaios físicos medidos. Informar ambiente, procedimento e evidência de cada afirmação.
9. Discussão. Analisar limitações, rastreamento, dependência de iluminação/rede, ausência de sensores e consequências da falta de fila.
10. Conclusão e trabalhos futuros. Relacionar o que o software já executa aos objetivos. Apresentar backend, persistência e sensores como próximas etapas.
11. Referências e apêndices. Incluir fontes verificadas, diagrama de arquitetura, esquema elétrico real, exemplos JSON e trechos relevantes do código.

Dados que a equipe precisa fornecer

Informar autores e contexto acadêmico; modelo exato da ESP32; câmera utilizada; dimensões e materiais da estrutura; fontes, tensões e correntes da montagem; método de fixação e alinhamento dos recipientes; custo e lista de componentes; fotos e esquema elétrico; condições de iluminação e rede; resultados de ensaios físicos com procedimento e amostra; regras de formatação solicitadas pela instituição.

Ensaios sugeridos ainda sem resultados

Planejar teste por cor com quantidades conhecidas de peças; repetição do mesmo objeto parado; variação de iluminação; peças próximas; reconexão; pausa manual; bloqueio térmico simulado; fechamento exato de lote; medição de latência entre detecção e parada física; repetibilidade do alinhamento. Registrar os resultados antes de afirmar desempenho.

11 Instrução pronta para a IA redatora

Use o texto a seguir junto com este documento e, se possível, com o código atual do projeto.

Você é uma IA responsável por elaborar o relatório técnico do projeto EcoCycle. Escreva em português brasileiro, com linguagem acadêmica clara e sem tom promocional. Este documento é a base factual do protótipo e descreve o estado do código em 29 de outubro de 2026.

Organize o relatório conforme a estrutura da seção 10. Explique as relações entre câmera, middleware Python, navegador, WebSocket e ESP32. Descreva o ciclo identificar, parar, posicionar e retomar, incluindo seleção do lote e início explícito. Informe que o firmware atual usa motor TT com L9110S e servo SG90 posicional de 180 graus, com plástico em 0 graus, alumínio em 90 graus e metal em 180 graus. Rejeito continua na identificação e simulação, sem comando próprio do servo.

Distinga rigorosamente três situações: funcionalidades implementadas no software; indicadores simulados; funcionalidades propostas. Peso, temperatura, força e velocidade são fictícios. O backend ainda não está integrado. A câmera classifica cores associadas aos materiais, sem identificar composição física. Não atribua ao projeto inteligência artificial treinada, sensores de peso/temperatura/velocidade, banco de dados, histórico ou mecanismos externos que não aparecem nos fontes.

Não invente resultados de acurácia, tempo de resposta, produtividade, torque, corrente, custos, número de peças testadas ou capacidade de carga. Apresente os 108 testes Python como evidência automatizada registrada na sessão, separada de qualquer validação mecânica. A presença de um teste C++ não autoriza afirmar que foi executado ou que o hardware foi validado. Quando faltarem dados, use [PREENCHER PELA EQUIPE] e indique qual evidência é necessária.

Use os contratos JSON da seção 7 como exemplos ilustrativos da comunicação implementada. Não os confunda com um contrato de backend já aprovado. Explique que ACK done confirma a sequência do código e que a telemetria representa o acionamento declarado, sem sensor físico confirmando posição ou movimento. Mencione as limitações de rastreamento e a necessidade de espaçamento entre peças porque não existe fila de materiais.

Não copie senhas, SSIDs reais, IPs particulares ou caminhos pessoais do computador. Referências bibliográficas precisam ser reais, verificadas e adequadas ao assunto; se não houver acesso às fontes, marque [REFERÊNCIA A VERIFICAR]. Não afirme conformidade com ABNT ou outra norma sem conhecer a exigência e conferir a formatação. Apresente a integração com backend e sensores reais na seção de trabalhos futuros.

Fontes locais prioritárias

Consultar config.py para valores atuais; app.py para rotas; vision.py, background.py, mjpeg.py e camera_buffer.py para visão; triage.py para operação; sorting.py e websocket_server.py para comandos; batch_weights.py, temperature.py e motion.py para simulações; esp32/EcoCycleClient/EcoCycleClient.ino para atuação; tests para regras verificadas. Usar GUIA_CODIGO.md, SIMULACAO_PESOS.md e esp32/README.md como apoio e conferir eventuais divergências no código atual.

O documento é um contexto de projeto e um roteiro de redação. A autoria, a montagem real e os resultados experimentais precisam ser confirmados pela equipe responsável.